

MINESWEEPER

Part 1: the board and its numbers

THE GAME

			1	0	1		
			2	1	1		
				1	1		
	2	2	2	2	1	1	
	1	0	1		1	1	1

Some cells hide mines. Gray cells haven't been revealed yet.

A revealed cell shows how many of its neighbors are mines.

Reveal a mine and you lose.

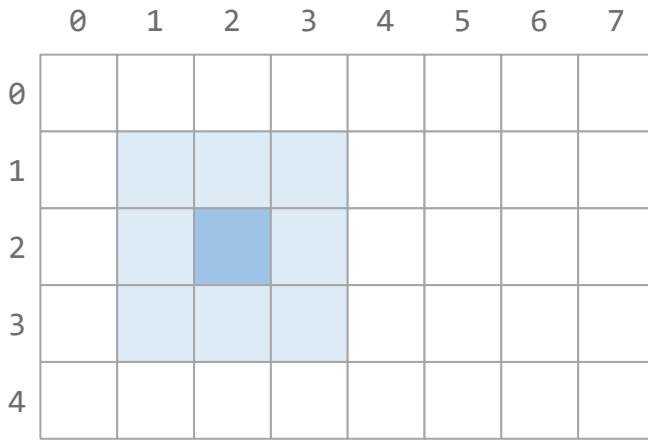
Reveal every cell that isn't a mine and you win.

BRAINSTORMING/PSEUDOCODE

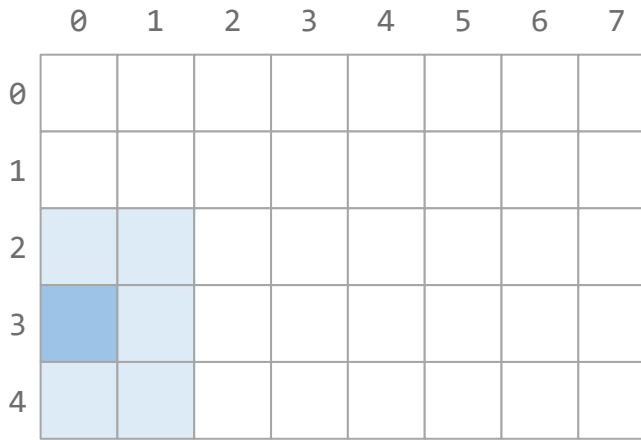
				1	0
				2	0
				2	0
		2	2	2	1
	1	0	1		
	1	0	1		

Let's say the user clicks on a square that doesn't have a mine on it. What do we need to do?

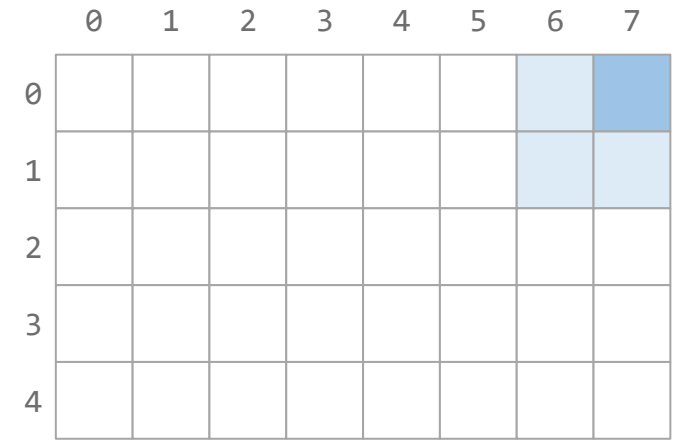
NEIGHBORS



Middle cell (2, 2): 8 neighbors



Edge cell (3, 0): 5 neighbors



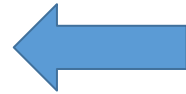
Corner cell (0, 7): 3 neighbors

Cells on the edge of the board have fewer neighbors. Don't look past the edge.

REPRESENTING THE BOARD

At the top of the file, outside any function

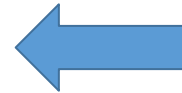
```
const int ROWS = 5;  
const int COLUMNS = 8;
```



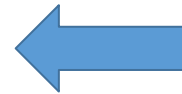
Global constants: every function can use them

Inside main

```
bool mines[ROWS][COLUMNS] = {};  
  
int adjacent_counts[ROWS][COLUMNS] = {};
```



true where there's a mine



How many mines touch each cell

To change the size of the board, change one line.

OUR TEST BOARD

```
bool mines[ROWS][COLUMNS] = {  
    {1, 0, 0, 0, 0, 0, 0, 0},  
    {0, 0, 1, 0, 0, 0, 1, 0},  
    {0, 1, 0, 1, 0, 0, 0, 0},  
    {0, 0, 0, 0, 0, 0, 0, 1},  
    {1, 0, 0, 0, 1, 0, 0, 0}  
};
```

In a bool array, 0 means false and 1 means true.

A fixed board means we know the right answers, so we can check our work.

row	column							
	0	1	2	3	4	5	6	7
0	*							
1			*				*	
2		*		*				
3								*
4	*				*			

TODAY'S FUNCTIONS

```
int count_mines(bool mines[][COLUMNS]);  
bool is_in_bounds(int row, int column);  
int count_adjacent_mines(bool mines[][COLUMNS], int row, int column);  
void compute_adjacent_counts(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS]);  
void print_debug_board(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS]);
```

In this order. Each one can use the ones before it.

COUNTING MINES

```
int count_mines(bool mines[][COLUMNS])
```

Returns how many cells in `mines` are true.

For the test board, that's 8.

YOUR TURN: count_mines

Write this function

```
int count_mines(bool mines[][COLUMNS])
```

Then put this in main, replacing any earlier test

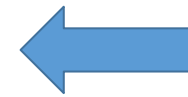
```
cout << count_mines(mines) << '\n';
```

Expected output

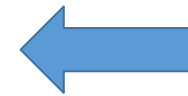
```
$ ./minesweeper  
8
```

ONE SOLUTION: count_mines

```
int count_mines(bool mines[][COLUMNS]) {  
    int count = 0;  
    for (int row = 0; row < ROWS; row++) {  
        for (int column = 0; column < COLUMNS; column++) {  
            if (mines[row][column]) {  
                count++;  
            }  
        }  
    }  
    return count;  
}
```



Visit every cell



Count the true ones

STAYING IN BOUNDS

```
bool is_in_bounds(int row, int column)
```

Returns true if (row, column) is on the board.

Call	Returns	Why
<code>is_in_bounds(0, 0)</code>	true	Top-left corner
<code>is_in_bounds(4, 7)</code>	true	Bottom-right corner
<code>is_in_bounds(-1, 2)</code>	false	Above the board
<code>is_in_bounds(5, 0)</code>	false	Below the board
<code>is_in_bounds(2, 8)</code>	false	Right of the board

C++ doesn't check indices, so we check them ourselves.

YOUR TURN: `is_in_bounds`

Write this function

```
bool is_in_bounds(int row, int column)
```

Then put this in main, replacing any earlier test

```
cout << is_in_bounds(0, 0) << '\n';    // top-left corner
cout << is_in_bounds(4, 7) << '\n';    // bottom-right corner
cout << is_in_bounds(-1, 2) << '\n';   // above the board
cout << is_in_bounds(5, 0) << '\n';    // below the board
cout << is_in_bounds(2, -1) << '\n';   // left of the board
cout << is_in_bounds(2, 8) << '\n';   // right of the board
```

Expected output

```
$ ./minesweeper
1
1
0
0
0
0
```

`cout` prints true as 1 and false as 0.

ONE SOLUTION: `is_in_bounds`

```
bool is_in_bounds(int row, int column) {  
    return row >= 0 && row < ROWS && column >= 0 && column < COLUMNS;  
}
```

true only when all four comparisons are true.

This does the same thing as an if/else that returns true or false.

COUNTING NEIGHBORS

```
int count_adjacent_mines(bool mines[][COLUMNS], int row, int column)
```

row	0	1	2	3	4	5	6	7	column
0	*								
1			*				*		
2		*		*					
3								*	
4	*				*				

Returns how many of the 8 cells around (row, column) are mines.

On the test board, the cell at (2, 2) has 3 neighboring mines.

OFFSETS

		column_offset		
		-1	0	1
row_offset	-1	-1, -1	-1, 0	-1, 1
	0	0, -1	cell	0, 1
	1	1, -1	1, 0	1, 1

Every neighbor is at:

```
int neighbor_row = row + row_offset;  
int neighbor_column = column + column_offset;
```

Each offset goes from -1 to 1.

When both offsets are 0, that's the cell itself.

THREE THINGS TO GET RIGHT

1. Loop each offset from -1 to 1:

```
for (int row_offset = -1; row_offset <= 1; row_offset++) {
```

2. Skip the cell itself, where both offsets are 0.

3. Check `is_in_bounds(neighbor_row, neighbor_column)` before you look at `mines[neighbor_row][neighbor_column]`.

YOUR TURN: count_adjacent_mines

Write this function

```
int count_adjacent_mines(bool mines[][COLUMNS], int row, int column)
```

Then put this in main, replacing any earlier test

```
cout << count_adjacent_mines(mines, 2, 2) << '\n';    // middle cell
cout << count_adjacent_mines(mines, 3, 0) << '\n';    // left edge
cout << count_adjacent_mines(mines, 0, 7) << '\n';    // corner cell
cout << count_adjacent_mines(mines, 0, 4) << '\n';    // top edge
cout << count_adjacent_mines(mines, 1, 2) << '\n';    // a mine itself
```

Expected output

```
$ ./minesweeper
3
2
1
0
2
```

ONE SOLUTION: count_adjacent_mines

```
int count_adjacent_mines(bool mines[][COLUMNS], int row, int column) {
    int count = 0;
    for (int row_offset = -1; row_offset <= 1; row_offset++) {
        for (int column_offset = -1; column_offset <= 1; column_offset++) {
            int neighbor_row = row + row_offset;
            int neighbor_column = column + column_offset;
            bool is_the_cell_itself = (row_offset == 0 && column_offset == 0);
            if (!is_the_cell_itself && is_in_bounds(neighbor_row, neighbor_column)) {
                if (mines[neighbor_row][neighbor_column]) {
                    count++;
                }
            }
        }
    }
    return count;
}
```

The first `if` skips the cell itself and anything off the board. Only then is it safe to look in `mines`.

WHEN THE NUMBERS ARE WRONG

What you see	Likely cause	Instead
The “a mine itself” test prints 3, not 2	Counting the cell itself	Skip it when both offsets are 0
Wrong numbers on edge cells, or a crash	No bounds check	Call <code>is_in_bounds</code> before reading mines
Numbers too low almost everywhere	Offset loops use <code>< 1</code>	<code><= 1</code> : the offsets are -1, 0, and 1
Numbers don’t match the mines around them	Row and column swapped	<code>mines[neighbor_row][neighbor_column]</code>

FILLING IN EVERY NUMBER

```
void compute_adjacent_counts(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS])
```

Fills adjacent_counts so every cell holds its number.

Call count_adjacent_mines once for each cell.

Mines get a number too. It just never gets shown.

YOUR TURN: compute_adjacent_counts

Write this function

```
void compute_adjacent_counts(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS])
```

Then put this in main, replacing any earlier test

```
int adjacent_counts[ROWS][COLUMNS] = {};  
compute_adjacent_counts(mines, adjacent_counts);  
cout << adjacent_counts[1][1] << '\n';  
cout << adjacent_counts[4][2] << '\n';
```

Expected output

```
$ ./minesweeper  
3  
0
```

ONE SOLUTION: compute_adjacent_counts

```
void compute_adjacent_counts(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS]) {  
    for (int row = 0; row < ROWS; row++) {  
        for (int column = 0; column < COLUMNS; column++) {  
            adjacent_counts[row][column] = count_adjacent_mines(mines, row, column);  
        }  
    }  
}
```

Same loops as `count_mines`. The work for each cell is one call to the function we already wrote.

PRINTING THE BOARD

```
void print_debug_board(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS])
```

Prints * for a mine and the number for every other cell, with a space after each.

One line per row. It shows everything, so we can check our work.

For the test board

```
$ ./minesweeper
* 2 1 1 0 1 1 1
2 3 * 2 1 1 * 1
1 * 3 * 1 1 2 2
2 2 2 2 2 1 1 *
* 1 0 1 * 1 1 1
```

YOUR TURN: print_debug_board

Write this function

```
void print_debug_board(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS])
```

Then put this in main, replacing any earlier test

```
int adjacent_counts[ROWS][COLUMNS] = {};  
compute_adjacent_counts(mines, adjacent_counts);  
print_debug_board(mines, adjacent_counts);
```

Expected output

```
$ ./minesweeper  
* 2 1 1 0 1 1 1  
2 3 * 2 1 1 * 1  
1 * 3 * 1 1 2 2  
2 2 2 2 2 1 1 *  
* 1 0 1 * 1 1 1
```

ONE SOLUTION: print_debug_board

```
void print_debug_board(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS]) {  
    for (int row = 0; row < ROWS; row++) {  
        for (int column = 0; column < COLUMNS; column++) {  
            if (mines[row][column]) {  
                cout << "* ";  
            } else {  
                cout << adjacent_counts[row][column] << " ";  
            }  
        }  
        cout << "\n";  
    }  
}
```

If your board prints as 8 rows of 5, with strange numbers at the bottom, the loop bounds are swapped: row goes with ROWS and column goes with COLUMNS.

RANDOM NUMBERS

Needs `#include <cstdlib>`

```
cout << rand() << '\n';  
cout << rand() << '\n';  
cout << rand() << '\n';
```

Output

```
$ ./random  
1804289383  
846930886  
1681692777
```

`rand()` returns a whole number from 0 up to a very large maximum.

Each call gives a different number.

RANDOM NUMBERS IN A RANGE

`rand() % 8` is the remainder after dividing by 8, so it's always 0 to 7.

So `rand() % ROWS` is always a valid row, and `rand() % COLUMNS` is always a valid column.

```
for (int i = 0; i < 4; i++) {  
    int row = rand() % ROWS;  
    int column = rand() % COLUMNS;  
    cout << row << ", " << column << '\n';  
}
```

Output

```
$ ./random  
3, 6  
2, 3  
3, 7  
1, 4
```

SEEDS

Run the first program twice:

First run

```
$ ./random  
1804289383  
846930886  
1681692777
```

Second run

```
$ ./random  
1804289383  
846930886  
1681692777
```

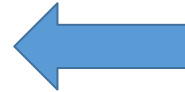
The same numbers! They're **pseudo-random**: calculated from a starting value called a **seed**.

Same seed, same numbers.

A DIFFERENT SEED EVERY TIME

Needs `#include <ctime>`

```
srand(time(0));  
cout << rand() % 100 << '\n';  
cout << rand() % 100 << '\n';  
cout << rand() % 100 << '\n';
```



`time(0)` is the current time, in seconds

First run

```
$ ./random  
52  
16  
8
```

Second run

```
$ ./random  
8  
64  
19
```

Call `srand` once, at the start of `main`.

SEEDS FOR TESTING

```
 srand(42);  
 cout << rand() % 100 << '\n';  
 cout << rand() % 100 << '\n';  
 cout << rand() % 100 << '\n';
```

A fixed seed gives the same “random” numbers every run.

Handy when you’re hunting a bug: the same board every time.

First run

```
$ ./random  
66  
40  
81
```

Second run

```
$ ./random  
66  
40  
81
```

Your numbers may differ from these. They depend on the compiler and system.

PLACING MINES

```
const int NUMBER_OF_MINES = 8;
```

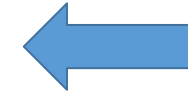
Pick a random row and column.

If there's already a mine there, try again.

Stop when NUMBER_OF_MINES mines are placed.

From a demo program that prints every pick

```
$ ./place_mines_demo
1, 3: mine 1
4, 6: mine 2
3, 1: mine 3
0, 3: mine 4
3, 1: already a mine, try again
4, 2: mine 5
4, 4: mine 6
3, 6: mine 7
2, 5: mine 8
```



Same cell twice

A RANDOM BOARD

The same run, printed with `print_debug_board`

```
$ ./place_mines_demo
0 0 2 * 2 0 0 0
0 0 2 * 3 1 1 0
1 1 2 1 2 * 2 1
1 * 2 2 2 4 * 2
1 2 * 2 * 3 * 2
```

Those same eight mines, with every number computed by the functions from this unit.

With `srand(time(0))`, every game gets a new board.

RANDOM MISTAKES

Mistake	What happens	Instead
<code>srand(time(0))</code> inside a loop	The same number over and over: 3 3 3 3 3	Call <code>srand</code> once, at the start of <code>main</code>
No <code>srand</code> at all	The same board every game	<code>srand(time(0))</code> at the start of <code>main</code>
<code>rand() % ROWS - 1</code>	-1 to 3: -1 is off the board, and row 4 is never picked	<code>rand() % ROWS</code>
<code>rand() % ROWS + 1</code>	1 to 5: 5 is off the board, and row 0 is never picked	<code>rand() % ROWS</code>

WHERE WE ARE

```
int count_mines(bool mines[][COLUMNS]);  
bool is_in_bounds(int row, int column);  
int count_adjacent_mines(bool mines[][COLUMNS], int row, int column);  
void compute_adjacent_counts(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS]);  
void print_debug_board(bool mines[][COLUMNS], int adjacent_counts[][COLUMNS]);
```

We can store a board and compute every number on it.

Next time: hide the board, let a player reveal cells, and place mines at random.